
Measuring and Improving Software Maintainability

A Static-Analysis-Driven Refactoring Lab Report

Pronob Karmoker

BSSE 1431

Subject: Software Maintainability & Refactoring Lab

Tools Used: Python 3, radon, pytest

1. Objective

This report documents a hands-on exercise in measuring and improving the maintainability of an existing, functionally correct code base (a Yahtzee dice-scoring module). Static analysis was performed with the **radon** library to establish a maintainability baseline, the source code was reviewed to identify structural weaknesses, the module was refactored into a new file, and the same static-analysis commands were re-run to quantify the improvement. Functional correctness was preserved and verified using the existing pytest suite plus an additional randomized cross-check.

2. Step A & B: Baseline Measurement and Code Review

The unmodified **yahtzee.py** file was analyzed with `radon cc yahtzee.py -s -a` (Cyclomatic Complexity) and `radon mi yahtzee.py -s` (Maintainability Index) before any changes were made. The raw tool output captured from the terminal is reproduced below as evidence of the baseline.

Baseline Cyclomatic Complexity (radon cc)

```
yahtzee.py
  M 200:4 Yahtzee.fullHouse - B (7)
  M 24:4  Yahtzee.ones - B (6)
  M 41:4  Yahtzee.twos - B (6)
  M 56:4  Yahtzee.threes - B (6)
  M 166:4 Yahtzee.smallStraight - B (6)
  M 183:4 Yahtzee.largeStraight - B (6)
  C 1:0   Yahtzee - A (5)
  M 14:4  Yahtzee.yahtzee - A (4)
  M 118:4 Yahtzee.two_pair - A (4)
  M 79:4  Yahtzee.fours - A (3)
  M 87:4  Yahtzee.fives - A (3)
  M 96:4  Yahtzee.sixes - A (3)
  M 104:4 Yahtzee.score_pair - A (3)
  M 138:4 Yahtzee.four_of_a_kind - A (3)
  M 152:4 Yahtzee.three_of_a_kind - A (3)
  M 4:4   Yahtzee.chance - A (1)
  M 71:4  Yahtzee.__init__ - A (1)
```

17 blocks analyzed.
Average complexity: A (4.12)

Baseline Maintainability Index (radon mi)

```
yahtzee.py - B (16.38)
```

```
radon raw yahtzee.py
LOC: 230  LLOC: 190  SLOC: 198  Comments: 0  Blank: 32
```

Reviewing the source alongside these numbers showed that although no single method was extremely complex in isolation, the Maintainability Index was pulled down to a **B (16.38)** — bordering on the "needs attention" range — by sheer size, heavy duplication, and low structural cohesion across the 230-line file.

3. Problem Identification

Three main issues were identified as the primary drivers of the low Maintainability Index:

- 1 **Severe code duplication.** Nearly every scoring method (ones, twos, threes, fours, fives, sixes, score_pair, two_pair, three_of_a_kind, four_of_a_kind, fullHouse) re-implements the same "count occurrences of each die face" logic from scratch, either as five repeated if-statements or a manually built count array. The same handful of ideas is written out 10+ times instead of once.
- 2 **Inconsistent, confusing API design.** The class mixes @staticmethod category functions that take five raw dice values (e.g. Yahtzee.ones(1,2,3,4,5)) with instance methods that require constructing an object first (e.g. Yahtzee(4,4,4,5,5).fours()), with no clear rule for which category uses which style. This makes the class harder to learn and use correctly.
- 3 **Poor naming and low-level, error-prone indexing.** Variables are named d1..d5, sum (which shadows the Python built-in), and cryptic tokens such as _2_at and _3_at. Face values are converted to array indices by hand (e.g. d1-1, 6-at-1) throughout the file instead of using a self-documenting data structure, which increases the risk of off-by-one bugs and makes the intent of each line hard to follow.

4. Step C: Refactoring Explanation

The code was rewritten into a new file, **clean_yahtzee.py**, applying the following clean-code strategies while keeping the exact original public API (method names and signatures) so that no calling code had to change:

- **Extracted shared helpers.** All duplicated tallying logic was collapsed into four small, pure functions built on collections.Counter: `_tally()`, `_score_face()`, `_n_of_a_kind()` and `_is_straight()`. Every category method now delegates to one of these instead of repeating the counting logic.
- **Eliminated magic-number indexing.** Manual index arithmetic (die-1, 6-at-1) was replaced with direct, readable Counter lookups keyed by the actual face value, and a named `FACES` range replaced hard-coded `range(6)` loops.
- **Improved naming and documentation.** Ambiguous names (sum, _2_at) were replaced with descriptive ones (pair_face, triple_face), and a module/class docstring now explains the preserved API contract.
- **Preserved backward compatibility on purpose.** The mixed static/instance API (fours/fives/sixes as instance methods, everything else as staticmethods) was intentionally kept unchanged so the existing test suite and any other caller continues to work without modification, isolating this exercise to internal code quality rather than a breaking interface redesign.
- **Removed dead and redundant code** such as unused local flags and duplicate loop variables that existed only as leftovers of the original implementation style.

5. Correctness Verification

Before trusting the metric improvement, functional equivalence was verified in two ways: (1) the original 15-test pytest suite (test_yahtzee.py) was pointed at clean_yahtzee.py and re-run, passing 15/15 with zero changes to the test assertions; and (2) an additional script generated 20,000 random five-dice rolls and compared the output of all 15 scoring categories between the legacy Yahtzee class and the refactored one, reporting **zero mismatches**. This gives strong confidence that behaviour was preserved exactly.

6. Step D: Post-Refactor Measurement

Cyclomatic Complexity (radon cc) — clean_yahtzee.py

```
clean_yahtzee.py
M 119:4 Yahtzee.fullHouse - B (7)
M 97:4  Yahtzee.two_pair - A (4)
F 36:0  _n_of_a_kind - A (3)
M 89:4  Yahtzee.score_pair - A (3)
F 45:0  _is_straight - A (2)
C 51:0  Yahtzee - A (2)
M 73:4  Yahtzee.yahtzee - A (2)
M 111:4 Yahtzee.smallStraight - A (2)
M 115:4 Yahtzee.largeStraight - A (2)
F 26:0  _tally - A (1)
F 31:0  _score_face - A (1)
M 54:4  Yahtzee.__init__ - A (1)
M 58:4  Yahtzee.fours - A (1)
M 61:4  Yahtzee.fives - A (1)
M 64:4  Yahtzee.sixes - A (1)
M 69:4  Yahtzee.chance - A (1)
M 77:4  Yahtzee.ones - A (1)
M 81:4  Yahtzee.twos - A (1)
M 85:4  Yahtzee.threes - A (1)
M 103:4 Yahtzee.three_of_a_kind - A (1)
M 107:4 Yahtzee.four_of_a_kind - A (1)
```

21 blocks analyzed.
Average complexity: A (1.86)

Maintainability Index (radon mi) — clean_yahtzee.py

clean_yahtzee.py - A (62.63)

radon raw clean_yahtzee.py
LOC: 125 LLOC: 77 SLOC: 71 Comments: 2 Blank: 31

7. Metrics Comparison Table

Metric	Before (yahtzee.py)	After (clean_yahtzee.py)	Change
Average Cyclomatic Complexity	A (4.12)	A (1.86)	-55%

Total Lines of Code (LOC)	230	125	-46%
Source Lines of Code (SLOC)	198	71	-64%
Maintainability Index (MI)	B (16.38)	A (62.63)	+282%
Blocks Analyzed	17	21	+4 (extracted helpers)

The refactor raised the Maintainability Index from a borderline **B (16.38)** to a healthy **A (62.63)**, while reducing total code size by nearly half and average cyclomatic complexity by more than half — all without changing observable behaviour.

8. Conclusion

This exercise demonstrates that maintainability problems are not always visible from correctness alone: the original module passed all functional tests yet scored poorly on maintainability due to duplication and structural inconsistency. By extracting shared logic into small, well-named helper functions and eliminating redundant counting code, the Maintainability Index improved from B to A and the codebase became substantially smaller and easier to reason about, while automated testing confirmed that no functional behaviour was altered in the process.